

CS T51 – OPERATING SYSTEMS

UNIT – III

System Model – Deadlock Characterization – Methods for handling Deadlocks - Deadlock Prevention – Deadlock avoidance – Deadlock detection – Recovery from Deadlocks - Storage Management – Swapping – Contiguous Memory allocation – Paging – Segmentation – Segmentation with Paging.

2- Mark Questions and Answers

1. Define Paging [Apr 2014 ,NOV 15, NOV 18]

It is a memory management scheme. This is another possible solution to the external fragmentation. This allow the physical address space of a process to be non-contiguous thus allowing a process to be allocated in physical memory where ever the free space is available.

2. What is External Fragmentation? [APR 2014]

External Fragmentation happens when a dynamic memory allocation algorithm allocates some memory and a small piece is left over that cannot be effectively used. If too much external fragmentation occurs, the amount of usable memory is drastically reduced. Total memory space exists to satisfy a request, but it is not contiguous.

3. What are conditions under which a deadlock situation may arise? [Apr 2014] [APR'15][NOV'14]

A deadlock situation can arise if the following four conditions hold simultaneously in a system:

- Mutual exclusion
- Hold and wait
- No pre-emption
- Circular-wait

4. What is a resource allocation graph?[APR'15, APR'17]

Deadlocks can be described more precisely in terms of a directed graph called a system resource allocation graph. This graph consists of a set of vertices V and a set of edges E . The set of vertices V is partitioned into two different types of nodes; P the set consisting of all active processes in the system and R the set consisting of all resource types in the system.

5. Define Swapping [APR'15, APR'16] [May 2019] [May 2018]

A process needs to be in memory to be executed. A process, however, can be swapped temporarily out of memory to a backing store, and then brought back into memory for continued execution. For example, assume a multiprogramming environment with a round-robin CPU-scheduling algorithm. When a quantum expires,

the memory manager will start to swap out the process that just finished, and to swap in another process to the memory space that has been freed.

6. Write four general strategies for dealing with deadlocks? (APR'15)(NOV'14) (NOV '17) (Nov' 2018) [May 2018]

A deadlock situation can arise if the following four conditions hold simultaneously in a system:

- Mutual exclusion
- Hold and wait
- No pre-emption
- Circular-wait

7. What is TLB? What is the advantage of TLB? (APR'17)

TLB stands for Translation Look aside Buffer. In a cached system, the base addresses of the last few referenced pages is maintained in registers called the TLB that aids in faster lookup. TLB contains those page-table entries that have been most recently used. Normally, each virtual memory reference causes 2 physical memory accesses one to fetch appropriate page table entry, and one to fetch the desired data.

Using TLB in between, this is reduced to just one physical memory access in cases of TLB hit. Problem with paging is that, extra memory references to access translation tables can slow programs down by a factor of 2 or 3. Too many entries in the translation tables to keep them all loaded in fast processor memory. The standard solution to this problem is to use a special, small fast lookup hardware cache; called TLB. The TLB is associative high speed memory.

8. Distinguish between preemptive and non-preemptive scheduling [Apr 2017]

Parameter	PREEMPTIVE SCHEDULING	NON-PREEMPTIVE SCHEDULING
Basic	In this resources(CPU Cycle) are allocated to a process for a limited time.	Once resources(CPU Cycle) are allocated to a process, the process holds it till it completes its burst time or switches to waiting state.
Interrupt	Process can be interrupted in between.	Process can not be interrupted until it terminates itself or its time is up.
Starvation	If a process having high priority frequently arrives in the ready queue, low priority process may starve.	If a process with long burst time is running CPU, then later coming process with less CPU burst time may starve.
Overhead	It has overheads of scheduling the processes.	It does not have overheads.
Flexibility	flexible	Rigid
Cost	cost associated	no cost associated
CPU Utilization	In preemptive scheduling, CPU utilization is high.	It is low in non preemptive scheduling.

Examples	Examples of preemptive scheduling are Round Robin and Shortest Remaining Time First.	Examples of non-preemptive scheduling are First Come First Serve and Shortest Job First.
----------	--	--

9. Why do deadlocks occur in Operating systems [Nov 2017]

Deadlock is a situation where a set of processes are blocked because each process is holding a resource and waiting for another resource acquired by some other process. Hold and Wait: A process is holding at least one resource and waiting for resources.

10. What is segmentation? [Nov 2017]

An important aspect of memory management that became unavoidable with paging is the separation of the user's view of memory and the actual physical memory. The user's view of memory is not the same as the actual physical memory. The user's view is mapped onto physical memory. The mapping allows differentiation between logical memory and physical memory.

11. What is segmentation with paging? [May 2019]

Both paging and segmentation have advantages and disadvantages. In fact, of the two most popular microprocessors now being used, the Motorola 68000 line is designed based on a flat-address space, whereas the Intel 80x86 and Pentium family are based on segmentation. Both are merging memory models toward a mixture of paging and segmentation. This combination is best illustrated by the architecture of the Intel 386. The IBM OS/2 32-bit version is an operating system running on top of the Intel 386 (and later) architecture. The 386 uses segmentation with paging for memory management. The maximum number of segments per process is 16 KB, and each segment can be as large as 4 gigabytes.

12. Under what circumstances do page faults occur? [Sep 2020]

A page fault occurs when an access to a page that has not been brought into main memory takes place. The operating system verifies the memory access, aborting the program if it is invalid.

A page fault occurs when a program attempts to access a block of memory that is not stored in the physical memory, or RAM. The fault notifies the operating system that it must locate the data in virtual memory, then transfer it from the storage device, such as an HDD or SSD, to the system RAM.

13. What is Safe State? [Sep 2020]

A state is safe if the system can allocate all resources requested by all processes (up to their stated maximums) without entering a deadlock state. All safe states are deadlock free, but not all unsafe states lead to deadlocks.

14. What are the advantages of dynamic loading? [MARCH 2021]

The advantage of dynamic loading is that an unused routine is never loaded. Dynamic loading does not require special support from the OS. Operating systems may help the programmer, however, by providing library routines to implement dynamic loading.

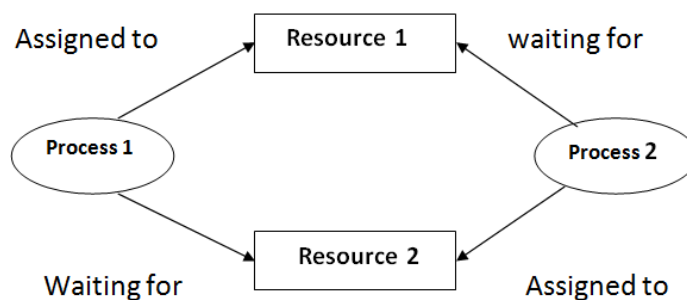
15. What is the use of overlays? [MARCH 2021]

Overlaying is a programming method that allows programs to be larger than the computer's main memory. An embedded system would normally use overlays because of the limitation of physical memory, which is internal memory for a system-on-chip, and the lack of virtual memory facilities.

Part 2**1. Explain Deadlock Prevention in detail? (APR'15) Explain how deadlocks are handled in Operating systems [Nov 2017] Describe bankers algorithms? (NOV '15) [Apr 2018]**

Deadlock: In the multiprogramming operating system, there are a number of processing which fights for a finite number of resources and sometimes waiting process never gets a chance to change its state because the resources for which it is waiting are held by another waiting process.

A set of a process is called deadlock when they are waiting for the happening of an event which is called by some another event in the same set.



Here every process will follow the system model which means the process requests a resource if not allocated then wait otherwise it allocated will use the resources and release it after use.

Methods for handling deadlock - There are mainly four methods for handling deadlock.

1. Deadlock ignorance

- It is the most popular method and it acts as if no deadlock and the user will restart.

- As handling deadlock is expensive to be called of a lot of codes need to be altered which will decrease the performance so for less critical jobs deadlock are ignored. Ostrich algorithm is used in deadlock Ignorance.
- Used in windows, Linux etc.

2. Deadlock prevention

It means that we design such a system where there is no chance of having a deadlock.

- **Mutual exclusion:**

- It can't be resolved as it is the hardware property.
- For example, the printer cannot be simultaneously shared by several processes.
- This is very difficult because some resources are not sharable.

- **Hold and wait:**

- Hold and wait can be resolved using the conservative approach where a process can start it and only if it has acquired all the resources.

- **Active approach:**

- Here the process acquires only requires resources but whenever a new resource requires it must first release all the resources.

- **Wait time out:**

- Here there is a maximum time bound until which a process can wait for other resources after which it must release the resources.

- **Circular wait:**

- In order to remove circular wait, we assign a number to every resource and the process can request only in the increasing order otherwise the process must release all the high number acquires resources and then make a fresh request.

- **No pre-emption:**

- In no pre-emption, we allow forceful pre-emption where a resource can be forcefully pre-empted.
- The pre-empted resource is added to the list of resources where the process is waiting.
- The new process can be restarted only when it regains its old resources. Priority must be given to a process which is in waiting for state.

3. Deadlock avoidance

- Here whenever a process enters into the system it must declare maximum demand. To the deadlock problem before the deadlock occurs.
- This approach employs an algorithm to access the possibility that deadlock would occur and not act accordingly.
- If the necessary condition of deadlock is in place it is still possible to avoid feedback by allocating resources carefully.

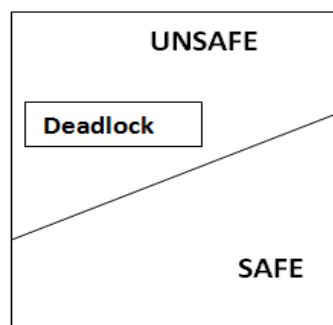
A **deadlock avoidance algorithm** dynamically examines the resources allocation state to ensure that a circular wait condition case never exists. Where the resources allocation state is defined by the of available and allocated resources and the maximum demand of the process.

There are 3 states of the system:

Safe state

When a system can allocate the resources to the process in such a way so that they still avoid deadlock then the state is called safe state. When there is a safe sequence exit then we can say that the system is in the safe state.

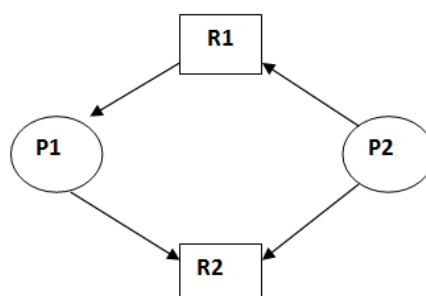
A sequence is in the safe state only if there exists a safe sequence. A sequence of process **P₁**, **P₂**, **P_n** is a safe sequence for the current allocation state if for each **P_i** the resources request that **P_i** can still make can be satisfied by currently available resources pulls the resources held by all **P_j** with $j < i$.



Methods for deadlock avoidance

1) Resource allocation graph

- This graph is also kind of graphical bankers' algorithm where a process is denoted by a circle P_i and resources is denoted by a rectangle R_j (.dots) inside the resources represents copies.
- Presence of a cycle in the resource's allocation graph is necessary but not sufficient condition for detection of deadlock. If the type of every resource has exactly one copy than the presence of cycle is necessary as well as sufficient condition for detection of deadlock.



This is in unsafe state (cycle exist) if P1 request P2 and P2 request R1 then deadlock will occur.

2) Bankers's algorithm

The resource allocation graph algorithms not applicable to the system with multiple instances of the type of each resource. So, for this system Banker's algorithm is used. Here whenever a process enters into the system it must declare maximum demand possible.

At runtime, we maintain some data structure like current allocation, current need, current available etc. Whenever a process requests some resources, we first check whether the system is in a safe state or not meaning if every process requires maximum resources then is there any sequence in which request can be entertaining if yes then request is allocated otherwise rejected.

Safety algorithm -This algorithm is used to find whether system is in safe state or not we can find:

Remaining Need = Max Need – Current allocation

Current available = Total available – Current allocation

Consider the following 3 process total resources are given for **A= 6, B= 5, C= 7, D = 6**

Process	maximum				Allocation				Need			
	A	B	C	D	A	B	C	D	A	B	C	D
P1	3	3	2	2	1	2	2	1	2	1	0	1
P2	1	2	3	4	1	0	3	3	0	2	0	1
P3	1	3	5	0	1	2	1	0	0	1	4	0

First we find the need matrix by **Need= maximum – allocation** .Then find available **resources = total – allocated**

A B C D(6 5 7 6) - A B C D(3 4 6 4), Available resources A B C D (3 1 1 2)

Then we check whether system is in deadlock or not & find safe sequence of process.

P1 can be satisfied: Available= P1 allocated + available (1,2,2,1) +(3,1,1,2) = (4,3,3, 3)

P2 can be satisfied: Available=P2 allocated +available (1,0,3,3) +(4,3,3,3) = (5,3,6,6)

P3 can be satisfied: Available=P3 allocated +available (1,2,1,0) +(5,3,6,6) = (6,5,7,6)

So, the system is safe and the safe sequence is **P1 → P2 → P3**

4. Detection and recovery

When the system is in deadlock then one method is to inform the operates and then operator deal with deadlock manually and the second method is system will automatically recover from deadlock. There are two ways to recover from deadlock:

- **Process termination:** Deadlock can be eliminated by aborting a process. Abort all deadlock process. Abort is processed at a time until the deadlock cycle is eliminated. This can help to recover the system from file deadlock.
- **Resources preemption:** To eliminate deadlock using resources preemption, we prompt the same resources pas processes and give these resources to another process until the deadlock cycle is broken. Here a process is partially rollback until the last checkpoint or and hen detection algorithm is executed.

2. Explain about contiguous memory allocation? (APR'14)

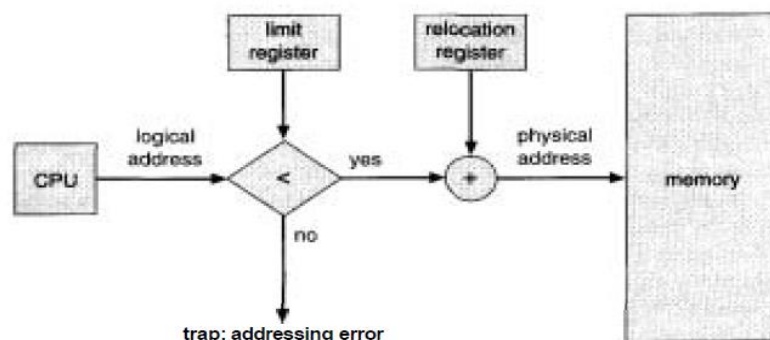
Contiguous Memory Allocation

The main memory must accommodate both the operating system and the various user processes. Therefore we need to allocate different parts of the main memory in the most efficient way possible. This section will explain one common method, contiguous memory allocation. The memory is usually divided into two partitions: one for the resident operating system, and one for the user processes. We may place the operating system in either low memory or high memory. In this contiguous memory allocation, each process is contained in a single contiguous section of memory.

Memory Protection

Memory protection is to protecting the operating system from user processes, and protecting user processes from one another, by using limit register and relocation register)

The relocation register contains the value of the smallest physical address; the limit register contains the range of logical addresses (for example, relocation = 100040 and limit = 74600). With relocation and limit registers, each logical address must be less than the limit register; the MMU maps the logical address *dynamically* by adding the value in the relocation register. This mapped address is sent to memory . When the CPU scheduler selects a process for execution, the dispatcher loads the relocation and limit registers with the correct values as part of the context switch.



Because every address generated by the CPU is checked against these registers, we can protect both the operating system and the other users programs and data from being modified by this running process.

The relocation-register scheme provides an effective way to allow the operating-system size to change dynamically. For example, the operating system contains code and buffer space for device drivers. If a device driver (or other operating-system service) is not commonly used, we do not want to keep the code and data in memory, as we might be able to use that space for other purposes. Such code is sometimes called **transient** operating-system code.

Memory Allocation

One of the simplest methods for memory allocation is to divide memory into several fixed-sized partitions. Each partition may contain exactly one process. Thus, the degree of multiprogramming is bound by the number of partitions. In this multiple-partition method, when a partition is free, a process is selected from the input queue and is loaded into the free partition. When the process terminates, the partition becomes available for another process.

Initially, all memory is available for user processes, and is considered as one large block of available memory, a hole. When a process arrives and needs memory, we search for a hole large enough for this process. If we find one, we allocate only as much memory as is needed, keeping the rest available to satisfy future requests.

This procedure is a particular instance of the general dynamic storage allocation problem, which is how to satisfy a request of size n from a list of free holes. There are many solutions to this problem. The set of holes is searched to determine which hole is best to allocate.

The first-fit, best-fit, and worst-fit strategies are the most common ones used to select a free hole from the set of available holes.

First fit:

Allocate the first hole that is big enough. Searching can start either at the beginning of the set of holes or where the previous first-fit search ended. We can stop searching as soon as we find a free hole that is large enough.

Best fit:

Allocate the smallest hole that is big enough. We must search the entire list, unless the list is kept ordered by size. This strategy produces the smallest leftover hole.

Worst fit:

Allocate the largest hole. Again, we must search the entire list, unless it is sorted by size. This strategy produces the largest leftover hole, which may be more useful than the smaller leftover hole from a best-fit approach.

Simulations have shown that both first fit and best fit are better than worst fit in terms of decreasing both time and storage utilization. Neither first fit nor best fit is clearly better in terms of storage utilization, but first fit is generally faster. These algorithms, however, suffer from external fragmentation.

3. Explain Paging in detail? (APR'14, NOV '15) [Apr 2018]

Paging

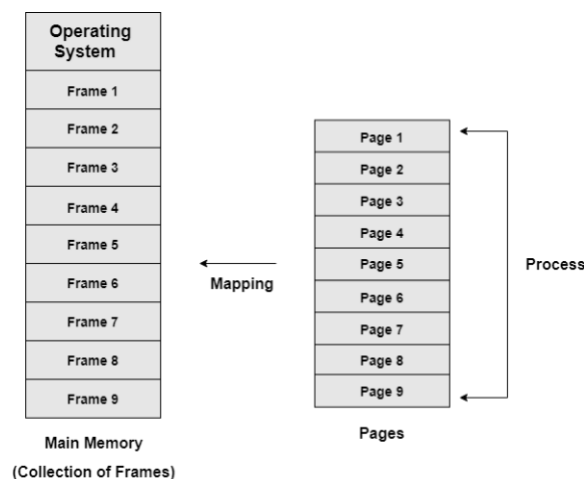
Paging is a memory-management scheme that permits the physical-address space of a process to be noncontiguous. Paging avoids the considerable problem of fitting the varying-sized memory chunks onto the backing store, from which most of the previous memory-management schemes suffered. Traditionally, support for paging has been handled by hardware.

In Operating Systems, Paging is a storage mechanism used to retrieve processes from the secondary storage into the main memory in the form of pages. The main idea behind the paging is to divide each process in the form of pages. The main memory will also be divided in the form of frames.

One page of the process is to be stored in one of the frames of the memory. The pages can be stored at the different locations of the memory but the priority is always to find the contiguous frames or holes.

Pages of the process are brought into the main memory only when they are required otherwise, they reside in the secondary storage.

Different operating system defines different frame sizes. The sizes of each frame must be equal. Considering the fact that the pages are mapped to the frames in Paging, page size needs to be as same as frame size.



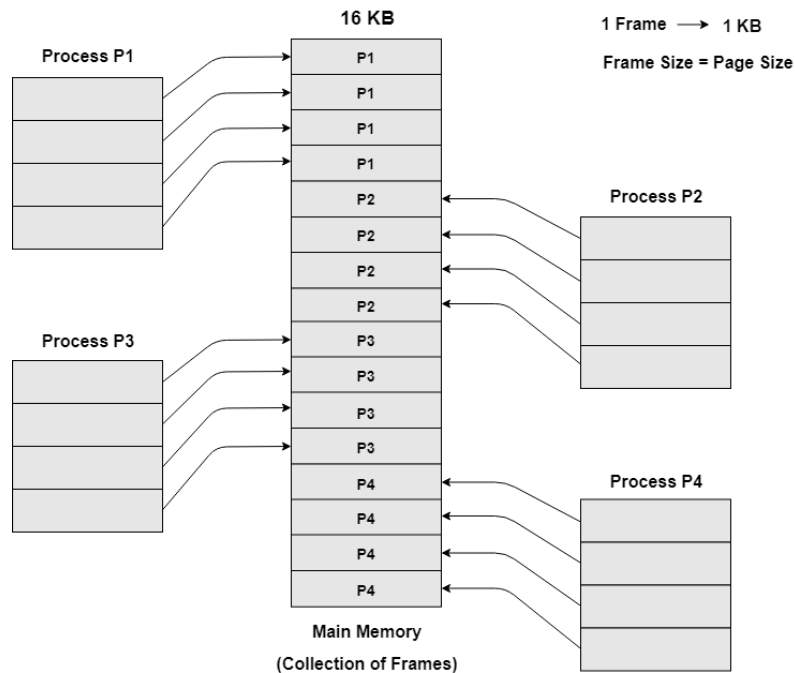
Example

Let us consider the main memory size 16 Kb and Frame size is 1 KB therefore the main memory will be divided into the collection of 16 frames of 1 KB each.

There are 4 processes in the system that is P1, P2, P3 and P4 of 4 KB each. Each process is divided into pages of 1 KB each so that one page can be stored in one frame.

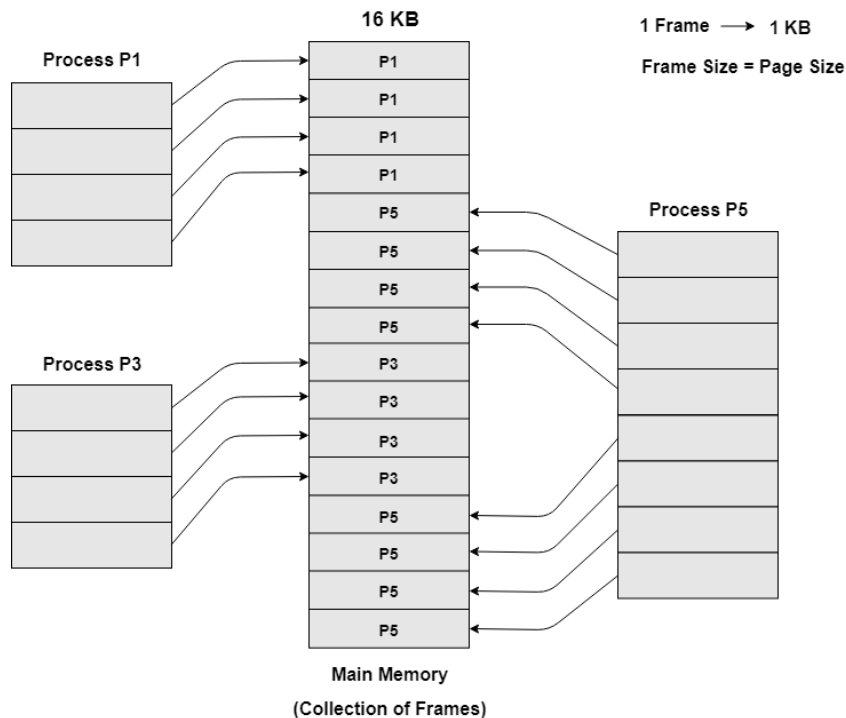
Initially, all the frames are empty therefore pages of the processes will get stored in the contiguous way.

Frames, pages and the mapping between the two is shown in the image below.



Let us consider that, P2 and P4 are moved to waiting state after some time. Now, 8 frames become empty and therefore other pages can be loaded in that empty place. The process P5 of size 8 KB (8 pages) is waiting inside the ready queue.

Given the fact that, we have 8 non contiguous frames available in the memory and paging provides the flexibility of storing the process at the different places. Therefore, we can load the pages of process P5 in the place of P2 and P4.



Memory Management Unit

The purpose of Memory Management Unit (MMU) is to convert the logical address into the physical address. The logical address is the address generated by the CPU for every page while the physical address is the actual address of the frame where each page will be stored.

When a page is to be accessed by the CPU by using the logical address, the operating system needs to obtain the physical address to access that page physically.

The logical address has two parts.

1. Page Number
2. Offset

Memory management unit of OS needs to convert the page number to the frame number.

Paging Protection

The paging process should be protected by using the concept of insertion of an additional bit called Valid/Invalid bit. Paging Memory protection in paging is achieved by associating protection bits with each page. These bits are associated with each page table entry and specify protection on the corresponding page.

Advantages of Paging

- Easy to use memory management algorithm
- No need for external Fragmentation
- Swapping is easy between equal-sized pages and page frames.

Disadvantages of Paging

- May cause Internal fragmentation
- Complex memory management algorithm
- Page tables consume additional memory.
- Multi-level paging may lead to memory reference overhead.

4. What are various address translation mechanism used in paging (APR 16)

Paging is a memory management scheme that eliminates the need for contiguous allocation of physical memory. This scheme permits the physical address space of a process to be non – contiguous.

- Logical Address or Virtual Address (represented in bits): An address generated by the CPU
- Logical Address Space or Virtual Address Space(represented in words or bytes): The set of all logical addresses generated by a program
- Physical Address (represented in bits): An address actually available on memory unit
- Physical Address Space (represented in words or bytes): The set of all physical addresses corresponding to the logical addresses

Example:

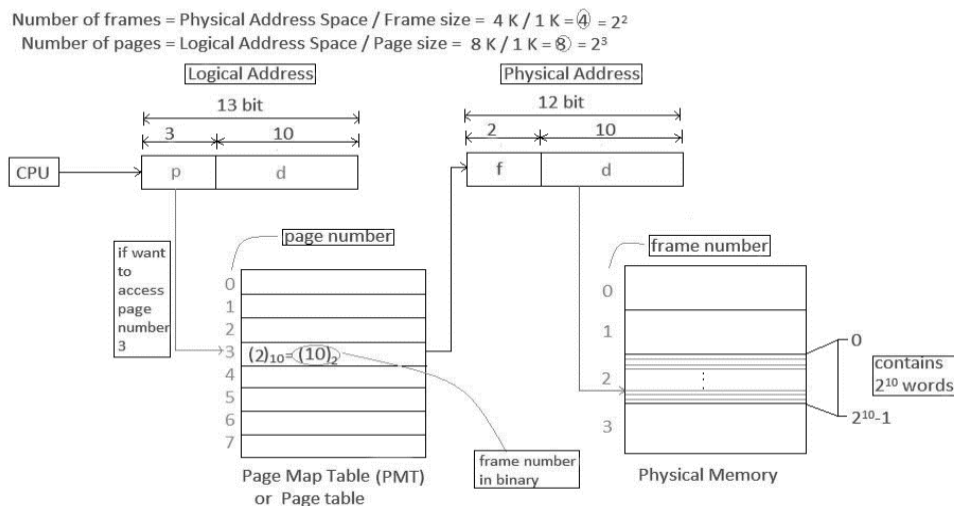
- If Logical Address = 31 bit, then Logical Address Space = 2^{31} words = 2 G words (1 G = 2^{30})
- If Logical Address Space = 128 M words = $2^7 * 2^{20}$ words, then Logical Address = $\log_2 2^{27} = 27$ bits
- If Physical Address = 22 bit, then Physical Address Space = 2^{22} words = 4 M words (1 M = 2^{20})
- If Physical Address Space = 16 M words = $2^4 * 2^{20}$ words, then Physical Address = $\log_2 2^{24} = 24$ bits

The mapping from virtual to physical address is done by the memory management unit (MMU) which is a hardware device and this mapping is known as paging technique.

- The Physical Address Space is conceptually divided into a number of fixed-size blocks, called **frames**.
- The Logical address Space is also splitted into fixed-size blocks, called **pages**.
- Page Size = Frame Size

Let us consider an example:

- Physical Address = 12 bits, then Physical Address Space = 4 K words
- Logical Address = 13 bits, then Logical Address Space = 8 K words
- Page size = frame size = 1 K words (assumption)



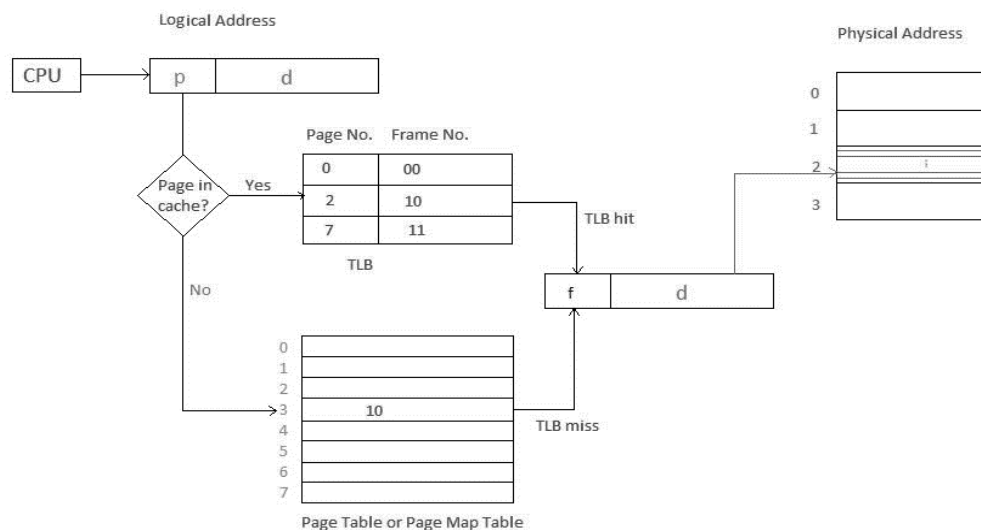
Address generated by CPU is divided into

- **Page number(p):** Number of bits required to represent the pages in Logical Address Space or Page number
- **Page offset(d):** Number of bits required to represent particular word in a page or page size of Logical Address Space or word number of a page or page offset.
- Physical Address is divided into

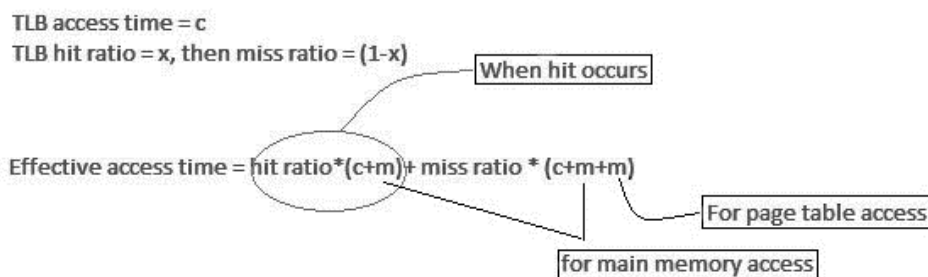
- **Frame number(f):** Number of bits required to represent the frame of Physical Address Space or Frame number.
- **Frame offset(d):** Number of bits required to represent particular word in a frame or frame size of Physical Address Space or word number of a frame or frame offset.

The hardware implementation of page table can be done by using dedicated registers. But the usage of register for the page table is satisfactory only if page table is small. If page table contain large number of entries then we can use TLB(translation Look-aside buffer), a special, small, fast look up hardware cache. The TLB is associative, high speed memory.

Each entry in TLB consists of two parts: a tag and a value. When this memory is used, then an item is compared with all tags simultaneously. If the item is found, then corresponding value is returned.



Main memory access time = m If page table are kept in main memory,
Effective access time = m(for page table) + m(for particular page in page table)



5. Compare the main memory organization schemes of contiguous memory allocation, pure segmentation, and pure paging with respect to the following issues: (7) (Nov '16)

(i) External fragmentation

(ii) Internal fragmentation

(iii) Ability to share code across processes

External fragmentation

- Contiguous Allocation with Fixed-Size Partitions: does not suffer from external fragmentation.
- Contiguous Allocation with Variable-Size Partitions: suffers from external fragmentation.
- Pure Segmentation: suffers from external fragmentation.
- Paging: does not suffer from external fragmentation.

Internal fragmentation

- Contiguous Allocation with Fixed-Size Partitions: suffers from internal fragmentation.
- Contiguous Allocation with Variable-Size Partitions: does not suffer from internal fragmentation.
- Pure Segmentation: does not suffer from internal fragmentation.
- Paging: suffers from internal fragmentation.

Ability to share code across processes

- Contiguous Allocation with Fixed-Size Partitions: no support for code sharing across processes.
- Contiguous Allocation with Variable-Size Partitions: no support for code sharing across processes
- Pure Segmentation: supports code sharing across processes. However, we must be careful to make sure that processes do not mix code and data in the same segment.
- Pure Paging: supports code sharing across processes. As in pure segmentation, we must make sure that processes do not mix code and data in the same page

6. Discuss the concept of segmentation with paging in Operating systems [Nov 2017] [Apr 2017]

Paging and Segmentation are the non-contiguous memory allocation techniques.

Paging divides the process into equal size partitions called as pages.

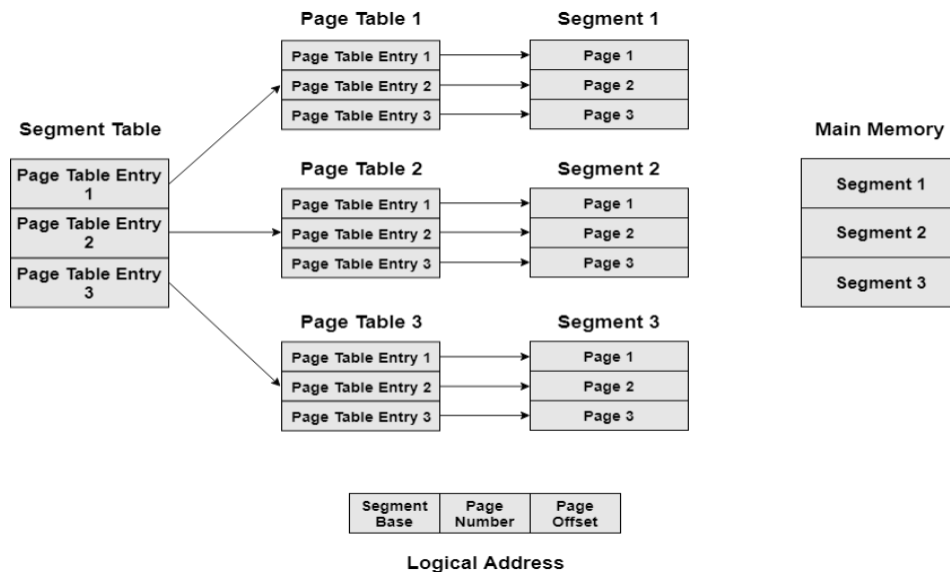
Segmentation divides the process into unequal size partitions called as segments.

Segmented Paging-Segmented paging is a scheme that implements the combination of segmentation and paging.

Page Number → It Points to the exact page within the segment

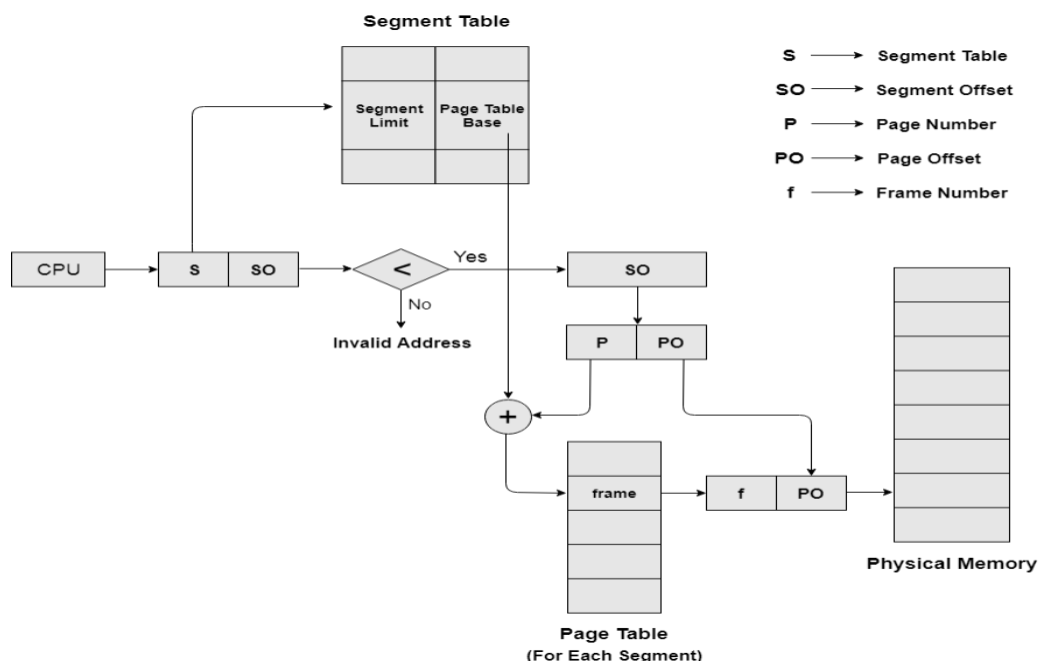
Page Offset → Used as an offset within the page frame

Each Page table contains the various information about every page of the segment. The Segment Table contains the information about every segment. Each segment table entry points to a page table entry and every page table entry is mapped to one of the page within a segment.



Translation of logical address to physical address

- The CPU generates a logical address which is divided into two parts: Segment Number and Segment Offset. The Segment Offset must be less than the segment limit. Offset is further divided into Page number and Page Offset. To map the exact page number in the page table, the page number is added into the page table base.
- The actual frame number with the page offset is mapped to the main memory to get the desired word in the page of the certain segment of the process.



Advantages of Segmented Paging

1. It reduces memory usage.
2. Page table size is limited by the segment size.
3. Segment table has only one entry corresponding to one actual segment.
4. External Fragmentation is not there.
5. It simplifies memory allocation.

Disadvantages of Segmented Paging

1. Internal Fragmentation will be there.
2. The complexity level will be much higher as compare to paging.
3. Page Tables need to be contiguously stored in the memory.

7. Explain segmentation with example [Nov 2018]**Segmentation**

In Operating Systems, Segmentation is a memory management technique in which, the memory is divided into the variable size parts. Each part is known as segment which can be allocated to a process.

The details about each segment are stored in a table called as segment table. Segment table is stored in one (or many) of the segments.

Segment table contains mainly two information about segment:

1. Base: It is the base address of the segment
2. Limit: It is the length of the segment.

Need for Segmentation

Paging is closer to Operating system rather than the User. It divides all the process into the form of pages regardless of the fact that a process can have some relative parts of functions which needs to be loaded in the same page.

Operating system doesn't care about the User's view of the process. It may divide the same function into different pages and those pages may or may not be loaded at the same time into the memory. It decreases the efficiency of the system.

It is better to have segmentation which divides the process into the segments. Each segment contain same type of functions such as main function can be included in one segment and the library functions can be included in the other segment,

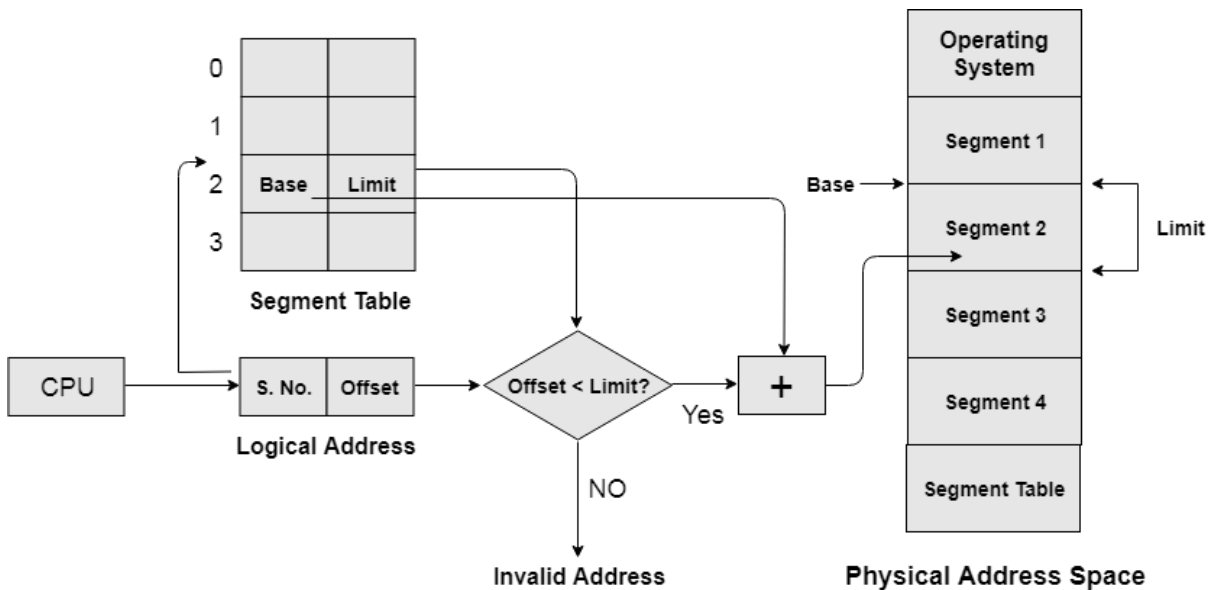
Translation of Logical address into physical address by segment table

CPU generates a logical address which contains two parts:

1. Segment Number
2. Offset

The Segment number is mapped to the segment table. The limit of the respective segment is compared with the offset. If the offset is less than the limit then the address is valid otherwise it throws an error as the address is invalid.

In the case of valid address, the base address of the segment is added to the offset to get the physical address of actual word in the main memory.



Advantages of Segmentation

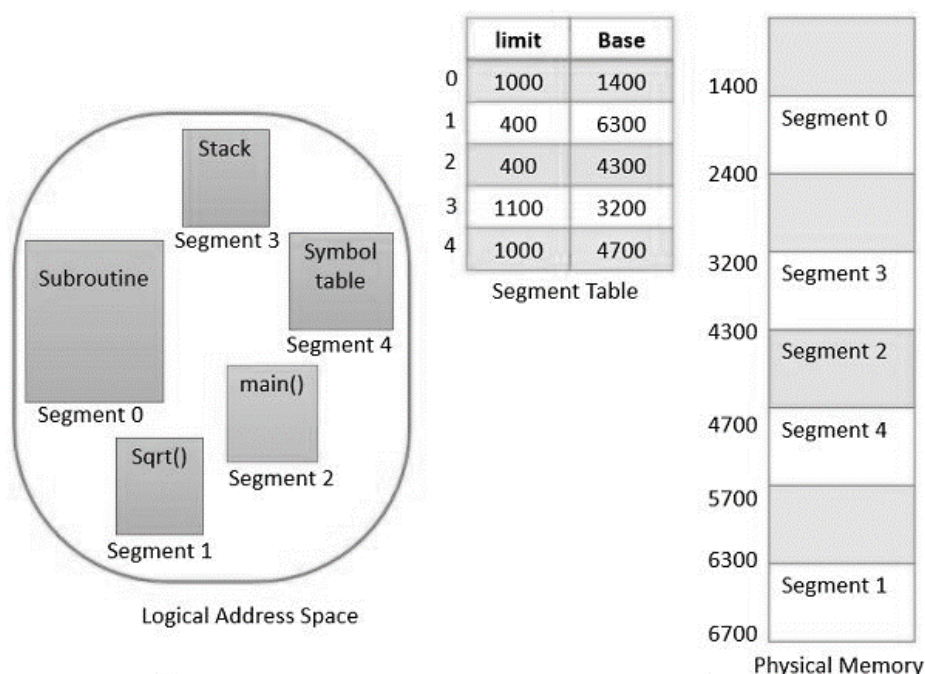
1. No internal fragmentation
2. Average Segment Size is larger than the actual page size.
3. Less overhead
4. It is easier to relocate segments than entire address space.
5. The segment table is of lesser size as compare to the page table in paging.

Disadvantages

1. It can have external fragmentation.
2. it is difficult to allocate contiguous memory to variable sized partition.
3. Costly memory management algorithms.

Example of Segmentation in OS

Consider that a user program has been divided into five segments and they are numbered from segment 0 to segment 4, as you can see them in the logical address space. You also have a segment table which has entries for these segments with their base address in physical memory and their limit.



Now suppose the CPU calls for segment number 2 which is 400 bytes long and it resides at 4300 memory location. The CPU wants to refer the 53rd byte of segment 2. So, here the input we get from CPU is segment number 2 and 53 as an offset.

Now the offset is in between 0 and limit of the segment 2 i.e. 400. So, the condition got verified and the offset is being added to the base address of the segment 2 to reach the 53rd byte of the segment 2 in physical memory. In case you try to access the 453rd byte of segment 2 then it would result in a trap to the operating system as offset value 453 is greater than the limit of segment 2.

8. Compare paging with segmentation with respect to the amount of memory required by the address translation structures in order to convert virtual addresses to physical addresses. Describe a mechanism by which on segment could belong to the address space of two different processes [May 2019]

S.NO	Paging	Segmentation
1.	In paging, program is divided into fixed or mounted size pages.	In segmentation, program is divided into variable size sections.
2.	For paging operating system is accountable.	For segmentation compiler is accountable.
3.	Page size is determined by hardware.	Here, the section size is given by the user.
4.	It is faster in the comparison of segmentation.	Segmentation is slow.
5.	Paging could result in internal fragmentation.	Segmentation could result in external fragmentation.

6.	In paging, logical address is split into page number and page offset.	Here, logical address is split into section number and section offset.
7.	Paging comprises a page table which encloses the base address of every page.	While segmentation also comprises the segment table which encloses segment number and segment offset.
8.	Page table is employed to keep up the page data.	Section Table maintains the section data.
9.	In paging, operating system must maintain a free frame list.	In segmentation, operating system maintain a list of holes in main memory.
10.	Paging is invisible to the user.	Segmentation is visible to the user.
11.	In paging, processor needs page number, offset to calculate absolute address.	In segmentation, processor uses segment number, offset to calculate full address.